

SEMANA 4 – SPRINT 1

Juan Luis Carrillo García

ProGest Análisis Inteligente

Macías Matus Jesús Daniel

**Universidad
Tecnológica de México**

12/06/2026

Contenido

PROYECTO INTEGRADOR DUAL	4
• Contexto Laboral (Simulación)	4
• Problema de Investigación	4
• Justificación	4
• Objetivo General.....	4
• Hipótesis.....	4
• Variables	4
• Métricas	5
• Alcance.....	5
• Arquitectura Preliminar	5
• Product Backlog Inicial	6
• Diseño Experimental	6
• Instrumentación y Métricas.....	6
• Cronograma	7
Introducción	8
Sistema Base: ProGest	8
Planteamiento del Problema.....	9
Justificación	9
Objetivo General.....	10
Objetivos Específicos	10
Hipótesis.....	11
MVP (Producto Mínimo Viable)	11
Variables.....	12
Variable Independiente.....	12
Variable Dependiente	12
Métricas	12
Métrica Principal	12
Métricas Secundarias	12

Diseño Experimental Preliminar	13
Referencias	14

PROYECTO INTEGRADOR DUAL

Título del Proyecto

Análisis Inteligente ProGest: Sistema Inteligente de Detección Temprana de Riesgo de Retraso en Tareas para la Gestión de Proyectos

- **Contexto Laboral (Simulación)**

La empresa ficticia ProGest Solutions utiliza una plataforma de gestión de proyectos. Se ha detectado que las tareas con riesgo de retraso suelen identificarse demasiado tarde. Se requiere un módulo inteligente que detecte riesgos de forma temprana y genere alertas preventivas.

- **Problema de Investigación**

La plataforma actual no identifica automáticamente tareas con alta probabilidad de retraso, limitando la capacidad de reacción de los líderes de proyecto.

- **Justificación**

El proyecto incorpora un componente lógico avanzado, instrumentación de métricas y validación empírica para medir mejoras reales.

- **Objetivo General**

Desarrollar e integrar un módulo inteligente de detección temprana de riesgo dentro de ProGest.

- **Hipótesis**

La implementación de un módulo de detección temprana de riesgo reducirá en al menos un 25% la cantidad de tareas entregadas fuera de plazo en comparación con la gestión tradicional sin alertas automáticas.

- **Variables**

VI: Uso del módulo Smart Risk Engine. VD: Porcentaje de tareas entregadas fuera de plazo.

- **Métricas**

TTFP= (Tareas retrasadas/Total tareas completadas) *100. Métricas secundarias: retraso promedio, alertas generadas, tareas por nivel de riesgo.

- **Alcance**

Incluye dashboard analítico, motor Smart Risk e instrumentación. Excluye aplicación móvil e integraciones externas.

- **Arquitectura Preliminar**

Frontend:

- *Next.js*
- *React*
- *Tailwind CSS*

Backend:

- *Flask*
- *API REST*
- *JWT Authentication*

Base de Datos:

- *PostgreSQL*

Componentes Existentes:

- *Gestión de usuarios*
- *Gestión de proyectos*
- *Gestión de tareas*
- *Comentarios*
- *Notificaciones*
- *Auditoría*

Nuevo Componente:

- *Smart Risk Engine*
- *Dashboard Analítico*
- *Módulo de Métricas*

• Product Backlog Inicial

ID	Historia de Usuario	Prioridad
HU-01	Como líder de proyecto quiero visualizar tareas con riesgo de retraso para tomar acciones preventivas.	Alta
HU-02	Como líder de proyecto quiero recibir alertas automáticas cuando una tarea alcance un nivel de riesgo alto o crítico.	Alta
HU-03	Como investigador quiero registrar métricas automáticamente para validar la hipótesis del proyecto.	Alta
HU-04	Como administrador quiero consultar indicadores históricos de desempeño.	Media
HU-05	Como usuario quiero visualizar un dashboard analítico con métricas relevantes.	Media
HU-06	Como líder de proyecto quiero consultar el historial de alertas generadas por el sistema.	Media
HU-07	Como administrador quiero exportar métricas para análisis estadístico.	Media
HU-08	Como usuario quiero visualizar el nivel de riesgo directamente en cada tarea.	Alta
HU-09	Como administrador quiero consultar la efectividad del motor de riesgo.	Baja
HU-10	Como investigador quiero comparar los resultados entre el grupo control y el grupo experimental.	Alta

• Diseño Experimental

Grupo control sin alertas visibles y grupo experimental con alertas y dashboard.

• Instrumentación y Métricas

Siguiendo la metodología de investigación aplicada establecida para el Proyecto Integrador Dual, el sistema generará automáticamente registros que permitirán evaluar el comportamiento de las tareas y validar la hipótesis planteada.

Para cada tarea se registrarán los siguientes datos:

Campo	Descripción
task_id	Identificador único de la tarea
project_id	Proyecto al que pertenece la tarea
Priority	Nivel de prioridad asignado
risk_score	Puntaje calculado por el Smart Risk Engine

risk_level	Clasificación de riesgo
created_at	Fecha de creación
started_at	Fecha de inicio
completed_at	Fecha de finalización
due_date	Fecha límite
delay_hours	Horas de retraso registradas
alert_generated	Indica si se generó una alerta
alert_date	Fecha de emisión de la alerta

Asimismo, se calcularán indicadores agregados como:

- Tasa de Tareas Fuera de Plazo (TTFP).
- Tiempo Promedio de Retraso.
- Cantidad de Alertas Generadas.
- Precisión del Sistema de Clasificación de Riesgo.
- Tiempo de Detección Temprana.

Toda esta información será almacenada automáticamente en la base de datos y utilizada posteriormente para el análisis estadístico de los resultados obtenidos.

• Cronograma

Sprint	Actividad Principal
Sprint 1	Investigación, hipótesis y planeación
Sprint 2	Diseño de arquitectura y base de datos
Sprint 3	Implementación del Smart Risk Engine
Sprint 4	Dashboard analítico
Sprint 5	Instrumentación y métricas
Sprint 6	Validación experimental y documentación

Introducción

La gestión eficiente de proyectos depende de la capacidad de identificar oportunamente los factores que pueden provocar retrasos en la ejecución de tareas. En muchos entornos de desarrollo de software, los líderes de proyecto detectan los problemas cuando las fechas límite ya han sido comprometidas, afectando la productividad del equipo y el cumplimiento de los objetivos establecidos.

Actualmente existen múltiples plataformas de gestión de proyectos que permiten administrar tareas, equipos y cronogramas; sin embargo, gran parte de estas herramientas se limita a mostrar información histórica sin proporcionar mecanismos preventivos que alerten de manera temprana sobre posibles incumplimientos.

Ante esta situación surge la necesidad de desarrollar una solución que permita anticipar riesgos mediante el análisis automatizado de la información disponible dentro del sistema.

Sistema Base: ProGest

ProGest es una plataforma SaaS orientada a la gestión de proyectos y equipos de trabajo. El sistema permite administrar proyectos, asignar tareas, gestionar miembros, realizar seguimiento de actividades y mantener comunicación entre los participantes mediante comentarios y notificaciones.

La plataforma fue desarrollada utilizando una arquitectura cliente-servidor, donde el frontend se encuentra implementado con Next.js y el backend mediante Flask, comunicándose a través de una API REST. La persistencia de datos se realiza mediante PostgreSQL y la seguridad se implementa utilizando autenticación basada en JWT y control de acceso por roles.

Entre sus funcionalidades principales se encuentran:

- Gestión de usuarios y autenticación.
- Gestión de proyectos.
- Administración de equipos de trabajo.
- Gestión de tareas y estados de avance.
- Sistema de comentarios.
- Notificaciones automáticas.
- Auditoría de acciones relevantes.

La elección de ProGest como base para este proyecto se debe a que ya cuenta con una arquitectura sólida y funcional, permitiendo enfocar el esfuerzo de desarrollo en la

incorporación de nuevas capacidades de análisis inteligente y generación de métricas para investigación aplicada.

Planteamiento del Problema

Los responsables de proyectos suelen identificar de forma tardía las tareas con alta probabilidad de retraso, lo que reduce la capacidad de reacción y aumenta la cantidad de entregables fuera de plazo.

La plataforma ProGest permite gestionar proyectos, tareas y equipos de trabajo; sin embargo, actualmente no cuenta con un mecanismo que analice automáticamente las condiciones de cada tarea para detectar riesgos potenciales antes de que se materialicen.

Esta limitación dificulta la toma de decisiones preventivas y afecta el cumplimiento de los cronogramas establecidos.

Justificación

La incorporación de mecanismos de análisis predictivo dentro de sistemas de gestión de proyectos puede mejorar la capacidad de planificación y seguimiento de los equipos de trabajo.

Desde la perspectiva de ingeniería de software, el proyecto permitirá extender una plataforma existente mediante la incorporación de un nuevo módulo inteligente basado en reglas de negocio y métricas de desempeño.

Desde la perspectiva de investigación aplicada, el proyecto permitirá evaluar mediante datos reales si las alertas tempranas de riesgo generan una reducción significativa en la cantidad de tareas entregadas fuera de plazo.

Objetivo General

Desarrollar e integrar un módulo inteligente de detección temprana de riesgo dentro de la plataforma ProGest que permita identificar tareas con probabilidad de retraso y evaluar su impacto sobre el cumplimiento de los plazos de entrega.

Objetivos Específicos

1. Analizar la arquitectura existente de ProGest para identificar componentes reutilizables.
2. Diseñar un algoritmo de evaluación de riesgo basado en reglas de negocio.
3. Implementar mecanismos de registro de métricas e instrumentación de datos.
4. Generar alertas tempranas para tareas con riesgo medio, alto o crítico.
5. Diseñar un tablero analítico para visualizar indicadores de desempeño.
6. Recolectar datos reales de uso mediante la operación del sistema.
7. Analizar estadísticamente los resultados obtenidos.
8. Validar o rechazar la hipótesis planteada.

Hipótesis

La implementación de un módulo de detección temprana de riesgo reducirá en al menos un 25% la cantidad de tareas entregadas fuera de plazo en comparación con la gestión tradicional sin alertas automáticas.

MVP (Producto Mínimo Viable)

El MVP del proyecto consistirá en la integración de un módulo denominado Smart Risk Engine dentro de la plataforma ProGest.

Este módulo tendrá como objetivo identificar tareas con alta probabilidad de retraso mediante un algoritmo basado en reglas de negocio y generar alertas preventivas para los responsables del proyecto.

El MVP incluirá las siguientes funcionalidades:

- Evaluación automática de riesgo para cada tarea.
- Clasificación de riesgo en niveles Bajo, Medio, Alto y Crítico.
- Dashboard analítico con indicadores de desempeño.
- Alertas visuales para tareas en riesgo.
- Registro automático de métricas para investigación.
- Exportación de resultados para análisis estadístico.

La funcionalidad será integrada sobre la arquitectura existente sin modificar los procesos principales de gestión de proyectos ya implementados en ProGest.

Variables

Variable Independiente

Uso del módulo Smart Risk Engine.

Valores:

- Sin Smart Risk.
- Con Smart Risk.

Variable Dependiente

Porcentaje de tareas entregadas fuera de plazo.

Métricas

Métrica Principal

Tasa de Tareas Fuera de Plazo (TTFP)

$TTFP = (Tareas\ retrasadas / Total\ de\ tareas\ completadas) \times 100$

Métricas Secundarias

- Tiempo promedio de retraso.
- Cantidad de alertas generadas.
- Cantidad de tareas clasificadas por nivel de riesgo.
- Tiempo promedio de detección anticipada.

Diseño Experimental Preliminar

Se utilizarán dos grupos de observación:

Grupo Control:

Utilizará el sistema sin visualizar las alertas de riesgo.

Grupo Experimental:

Utilizará el sistema con acceso al módulo Smart Risk y al tablero analítico.

Durante el periodo de evaluación se registrarán métricas operativas generadas automáticamente por el sistema para comparar el comportamiento de ambos grupos.

Posteriormente se realizará un análisis estadístico comparativo para determinar si existe una reducción significativa en la cantidad de tareas entregadas fuera de plazo.

Referencias

1. Project Management Institute. (2021). *A guide to the project management body of knowledge (PMBOK® Guide)* (7th ed.). Project Management Institute.
2. Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.
3. Sommerville, I. (2020). *Software engineering* (10th ed.). Pearson.
4. Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*. Scrum.org.
<https://scrumguides.org>
5. Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2nd ed.). Addison-Wesley.